

JUGEND FORSCHT 2027

Regionalwettbewerb Nordrhein-Westfalen (Ruhrgebiet)

Fachgebiet: Technik

MIRA

Machine Intelligence for Recycling Automation

Entwicklung und Evaluation eines ressourceneffizienten
Softwareprototyps zur Wertstofferkenennung

Jungforscher:	Jeremy Darko
Jahrgangsstufe:	Stufe EF (Jahrgangsstufe 11)
Schule:	Gymnasium Broich
Schulort:	Mülheim an der Ruhr
Datum der Einreichung:	15. Januar 2027

Projektüberblick

MIRA ist ein Softwareprototyp, der Glas, Metall, Papier, Kunststoff und Restmüll in Kamerabildern erkennen und lokalisieren soll. Untersucht wird, welcher messbare Zielkonflikt zwischen Erkennungsqualität, Modellgröße und CPU-Latenz entsteht, wenn ein kompaktes Detektionsmodell trainiert und auf INT8 quantisiert wird. Ein erster, abgeschlossener Machbarkeitsnachweis (Stage A) verglich Bildklassifikatoren für vier Wertstoffklassen. Historische Detektionsläufe EXP-013 bis EXP-017 verwendeten anschließend YOLO11n und unterschiedliche Mischungen öffentlicher Datensätze; ihre Ergebnisse dienen als Ausgangspunkt, stammen aber nicht aus dem neu aufgebauten Datensatz. Dieser aktuelle, annähernd nach Objektannotationen ausgeglichene Fünfklassen-Datensatz umfasst getrennte Trainings-, Validierungs- und Testsätze. Der zugehörige Trainings- und Quantisierungsvergleich steht noch aus. Ein Raspberry Pi, eine Kamera und ein Sortierarm sind als spätere Integrationsstufe geplant und wurden in der vorliegenden Softwareuntersuchung noch nicht vermessen.

Inhaltsverzeichnis

Projektüberblick	ii
Fachliche Kurzfassung	1
1 Einleitung und Motivation	2
1.1 Problemstellung	2
1.2 Ziel und Forschungsfrage	2
1.3 Themenfindung	2
1.4 Abgrenzung der Projektphasen	2
1.5 Aufbau der Arbeit	2
2 Theoretische Grundlagen	3
2.1 Lokale Inferenz und Zielkonflikt	3
2.2 Convolutional Neural Networks (CNNs)	3
2.3 Transfer Learning und MobileNetV2	3
2.4 Post-Training-Quantisierung (INT8)	3
2.5 Echtzeit-Objekterkennung mit You Only Look Once (YOLO11n)	4
2.6 Metriken der Detektion	4
3 Methodik und Experimentdesign	4
3.1 Stage A: abgeschlossener Klassifikationsvorversuch	4
3.2 Historische Detektionsentwicklung	4
3.3 Aktueller balancierter Fünfklassen-Datensatz	5
3.4 Geplanter kontrollierter Vergleich	5
4 Implementierung	6
4.1 Konfiguration und Kommandozeile	6
4.2 Datensatzaufbereitung	6
4.3 YOLO-Inferenz und Tracking	6
4.4 Dashboard-Prototyp	7
4.5 Modellexport	7
4.6 Nicht implementierte Systemteile	7
5 Ergebnisse	7
5.1 Historische Klassifikationsergebnisse	7
5.2 Historische Detektionsergebnisse	8
5.3 Einordnung des Field-Benchmarks	9
6 Diskussion	9
6.1 Bewertung der ursprünglichen Hypothesen	9
6.2 Gültigkeit der Auswertung	10
6.3 Klassen, Materialbegriff und Datenbias	10
6.4 Stand der laufenden Arbeiten	11

7 Fazit und Ausblick	11
7.1 Fazit	11
7.2 Ausblick	12
Quellen- und Literaturverzeichnis	13
Unterstützungsleistungen	15

Fachliche Kurzfassung

Diese Arbeit entwickelt einen Softwareprototyp zur kamerabasierten Erkennung von Wertstoffen. Die Forschungsfrage lautet: *Welcher messbare Zielkonflikt zwischen Detektionsgüte, Modellgröße und CPU-Inferenzlatenz ergibt sich beim Training und bei der INT8-Quantisierung eines kompakten YOLO11n-Modells für fünf Abfallklassen?*

Stage A untersuchte zuvor die einfachere Vierklassen-Klassifikation. Ein auf ImageNet vortrainiertes und feinabgestimmtes MobileNetV2 erreichte auf dem dortigen Validierungssplit 87.42 % Accuracy. Die Dateien umfassen 23.48 MiB für das Keras-Quellmodell, 8.49 MiB für den FP32-TFLite-Export und 2.61 MiB für den INT8-TFLite-Export; zwischen FP32- und INT8-TFLite lagen 0.07 Prozentpunkte Differenz in der Validierungsgenauigkeit (Yosinski u. a., 2014; Krishnamoorthi, 2018). Die historischen Detektionsläufe EXP-013 bis EXP-017 verwendeten andere Datensatzmischungen; ihr höchster dokumentierter mAP50-Wert beträgt 60.7 % (EXP-014). Diese Zahl ist kein Ergebnis des aktuellen Datensatzes.

Für die aktuelle Untersuchung wurde ein neuer Fünfklassen-Datensatz aus TACO, Roboflow Trash Detection, einem SAM-annotierten TrashNet-Bestand und dem öffentlichen dmedhi-Datensatz aufgebaut (Proença und Simões, 2020; Thung und Yang, 2016; Kirillov u. a., 2023; Roboflow Universe user jerry-jukbu, 2026; dmedhi, 2026). Er umfasst getrennte Trainings-, Validierungs- und Testdaten sowie ein Herkunftsmanifest. Der neue Trainingslauf, die Quantisierung und Messungen auf identischer CPU stehen noch aus. Raspberry-Pi- und Robotertests sind geplant, aber nicht Bestandteil der bisher gemessenen Ergebnisse.

1 Einleitung und Motivation

1.1 Problemstellung

Getrennt erfasste Abfälle können nur dann gezielt weiterverarbeitet werden, wenn Materialarten zuverlässig unterschieden werden. Kamerabasierte Objekterkennung ist dafür ein möglicher Baustein: Sie liefert zu einer Klasse auch die Position eines Gegenstands im Bild. Visuell ähnliche oder verformte Gegenstände, wechselnde Hintergründe und begrenzte Rechenleistung erschweren jedoch den Einsatz. MIRA untersucht dieses Problem zunächst als reproduzierbaren Softwareprototyp; Aussagen über die Sortierleistung einer vollständigen Anlage sind damit noch nicht möglich.

1.2 Ziel und Forschungsfrage

Ziel ist eine nachvollziehbare Pipeline für fünf Klassen: *glass*, *metal*, *paper*, *plastic* und *trash*. Im Mittelpunkt steht nicht der Nachweis eines fertigen Sortierroboters, sondern die folgende Forschungsfrage:

Welcher messbare Zielkonflikt zwischen Detektionsgüte, Modellgröße und CPU-Latenz ergibt sich beim Training und bei der INT8-Quantisierung eines kompakten YOLO11n-Modells für fünf Abfallklassen?

Die Detektionsgüte wird mit mAP50–95 als Hauptmetrik sowie mAP50, Precision, Recall und dem schwellenwertabhängigen F_1 -Score beschrieben. Modellgröße und Latenz werden für dasselbe exportierte Modell erfasst. Messungen auf einem Raspberry Pi und die Kopplung an einen Roboterarm bilden eine spätere Projektstufe.

1.3 Themenfindung

Die Projektidee entstand aus Beobachtungen der Mülltrennung im schulischen Alltag und bei Besuchen lokaler Recyclinghöfe. Daraus entwickelte sich die technische Frage, wie weit eine kleine, lokal ausführbare Computer-Vision-Pipeline mit dokumentierten Daten und Messgrößen kommt.

1.4 Abgrenzung der Projektphasen

Stage A ist ein abgeschlossener Machbarkeitsnachweis zur Klassifikation einzelner Bilder in vier Klassen. Die späteren EXP-013 bis EXP-017 sind historische Fünfklassen-Detektionsläufe mit unterschiedlichen Datenmischungen. Sie motivieren die aktuelle Untersuchung, ersetzen wegen ihrer unterschiedlichen Datengrundlagen aber keinen kontrollierten Vergleich. Der neu aufgebaute, annähernd ausgeglichene Datensatz und der darauf geplante Lauf bilden den aktuellen Arbeitsstand. Raspberry Pi, Kameraaufbau und Roboterarm sind noch nicht experimentell integriert.

1.5 Aufbau der Arbeit

Nach den notwendigen Grundlagen werden Datensätze, Trennregeln und Messgrößen beschrieben. Anschließend folgen Implementierung, bisherige Ergebnisse, deren Grenzen und die geplanten nächsten

Versuche.

2 Theoretische Grundlagen

2.1 Lokale Inferenz und Zielkonflikt

Edge AI bezeichnet hier die lokale Modellausführung auf einem Gerät mit begrenztem Speicher und begrenzter Rechenleistung (Deng u. a., 2020; Abadade u. a., 2023). Kleinere oder ganzzahlig quantisierte Modelle benötigen gewöhnlich weniger Speicher; eine geringere numerische Präzision kann jedoch Vorhersagen verändern. Deshalb werden Detektionsgüte, Dateigröße und Latenz getrennt gemessen. Eine niedrige Latenz auf einer Entwicklungs-CPU belegt noch keine entsprechende Leistung auf einem Raspberry Pi.

2.2 Convolutional Neural Networks (CNNs)

CNNs verarbeiten Bilder mit lernbaren Filtern und nutzen deren räumliche Struktur (LeCun, Bengio und Hinton, 2015; Bengio, Courville und Vincent, 2013). Ein Klassifikator gibt eine Klasse für das Gesamtbild aus. Ein Detektor sagt dagegen für jedes gefundene Objekt Klasse, Konfidenz und Begrenzungsrahmen voraus. Diese Unterscheidung ist wichtig, weil Accuracy aus Stage A nicht mit Detektionsmetriken aus Stage B verglichen werden kann.

2.3 Transfer Learning und MobileNetV2

Beim Transfer Learning werden vortrainierte Merkmale als Ausgangspunkt für eine neue Aufgabe genutzt (Yosinski u. a., 2014). Stage A verglich dazu ein von Grund auf trainiertes CNN mit MobileNetV2. MobileNetV2 reduziert Rechenaufwand unter anderem durch tiefenweise separierbare Faltungen und verwendet invertierte Residualblöcke (Sandler u. a., 2018). Diese Klassifikationsversuche sind ein Vorversuch; die aktuelle Forschungsfrage betrifft den Detektor.

2.4 Post-Training-Quantisierung (INT8)

Bei der Post-Training-Quantisierung werden Gleitkommawerte durch skalierte Ganzzahlen dargestellt. Vereinfacht gilt

$$q = \text{round} \left(\frac{r}{S} \right) + Z \quad (1)$$

mit Realwert r , Skala S und Nullpunkt Z (Krishnamoorthi, 2018). Kalibrierungsdaten bestimmen geeignete Wertebereiche. Rundung und Begrenzung können die Detektionen verändern; eine kleinere Datei allein belegt daher weder höhere Geschwindigkeit noch gleichbleibende Güte. Beides muss auf denselben Daten und derselben Hardware geprüft werden.

2.5 Echtzeit-Objekterkennung mit You Only Look Once (YOLO11n)

YOLO formuliert Objekterkennung als einstufige Vorhersage: Ein Netz erzeugt aus Merkmalen mehrerer Auflösungen Klassenwerte und Begrenzungsrahmen; anschließend werden konkurrierende Rahmen gefiltert (Redmon u. a., 2016; Neubeck und Van Gool, 2006). Die verbreitete Erklärung über ein festes $S \times S$ -Raster beschreibt vor allem die erste YOLO-Generation und ist keine hinreichende Beschreibung von YOLO11n. MIRA verwendet YOLO11n über die Ultralytics-Software (Jocher und Qiu, 2024). Die Bezeichnung „n“ kennzeichnet die kleinste Modellvariante der Familie, sagt aber noch nichts über ihre Eignung für die konkrete Zielhardware aus.

2.6 Metriken der Detektion

Eine Vorhersage zählt bei festgelegter Konfidenz- und IoU-Schwelle als richtig, wenn Klasse und räumliche Überlappung mit einer Referenzannotation passen. Precision beschreibt den Anteil richtiger unter den ausgegebenen Detektionen, Recall den Anteil gefundener Referenzobjekte. Der F_1 -Score ist ihr harmonisches Mittel und hängt von der gewählten Konfidenzschwelle ab:

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2)$$

Average Precision (AP) fasst die Precision-Recall-Kurve einer Klasse zusammen; mAP mittelt über Klassen. mAP50 nutzt eine IoU-Schwelle von 0,5. mAP50–95 mittelt zusätzlich über IoU-Schwellen von 0,50 bis 0,95 und bewertet die Lokalisierung strenger. Werte aus verschiedenen Testbeständen sind nicht ohne Weiteres direkt vergleichbar.

3 Methodik und Experimentdesign

3.1 Stage A: abgeschlossener Klassifikationsvorversuch

Stage A verwendete 796 unter kontrollierten Bedingungen aufgenommene Bilder der Klassen *glass*, *metal*, *paper* und *plastic*. Mit Seed 123 wurden 80 % für Training und 20 % für Validierung vorgesehen (etwa 637/159 Bilder). Verglichen wurden ein kleines CNN, MobileNetV2 mit eingefrorenem Backbone, Fine-Tuning und der INT8-Export (Sandler u. a., 2018; TensorFlow Authors, 2024). Eingabegrößen und Vorverarbeitung waren innerhalb des jeweiligen Modellvergleichs festgelegt. Stage A bewertet Bildklassifikation mit Accuracy; ihre Resultate sind nicht mit mAP-Werten der Detektion gleichzusetzen.

3.2 Historische Detektionsentwicklung

Für die Objekterkennung werden Bounding Boxes benötigt. Ein früher Auto-Labeling-Versuch mit Canny-Kanten und Otsu-Schwellenwert erfasste häufig kontrastreiche Tischbereiche statt der Gegenstände. Das zugehörige EXP-005 war deshalb kein belastbarer Nachweis der Generalisierung. Danach wurden unter anderem TACO, Roboflow Trash Detection und ein mithilfe von SAM nachannotierter TrashNet-Bestand eingesetzt (Proença und Simões, 2020; Roboflow Universe user jerry-jukbu, 2026; Thung und Yang, 2016; Kirillov u. a., 2023).

Die YOLO11n-Läufe EXP-013 bis EXP-017 sind historische Experimente. EXP-013 nutzte TACO und TrashNet, EXP-014 zusätzlich Roboflow, EXP-015 TACO, TrashNet und WaRP, EXP-016 nur WaRP und EXP-017 alle vier Quellen (Yudin u. a., 2024; Jocher und Qiu, 2024). Sie liefen jeweils 120 Epochen mit 640×640 Pixeln. Da sich Zusammensetzung und resultierende Validierungsbestände änderten, werden ihre mAP-Werte deskriptiv berichtet, aber nicht als kontrollierte Ablation oder als Resultat des aktuellen Datensatzes interpretiert.

3.3 Aktueller balancierter Fünfklassen-Datensatz

Der aktuelle Bestand merged_mira_balanced enthält 6898 Bilder: 5108 Training, 415 Validierung und 1375 Test. Alle Quellklassen werden programmatisch auf *glass*, *metal*, *paper*, *plastic* und *trash* abgebildet. Die Quellen und Splitregeln sind:

- TACO: deterministische Aufteilung 70/15/15 mit Seed 42 (Proença und Simões, 2020);
- Roboflow Trash Detection: Übernahme der veröffentlichten Train/Valid/Test-Zuordnung (Roboflow Universe user jerry-jukbu, 2026);
- SAM-annotiertes TrashNet: vorhandene Train/Val-Zuordnung (Thung und Yang, 2016; Kirillov u. a., 2023);
- dmedhi Garbage Image Classification/Detection: Übernahme der veröffentlichten Train/Validation-Zuordnung (dmedhi, 2026).

Das Training wird aus den jeweiligen Trainingsanteilen gebildet. Ein deterministischer Auswahlalgorithmus mit Seed 42 nähert die Klassen nach Anzahl der Bounding Boxes bis zu einem Ziel von höchstens 1800 Boxen je Klasse an. Da Bilder mehrere Objekte enthalten können und Dubletten anschließend entfernt werden, ist das Ergebnis annähernd, nicht exakt, ausgeglichen. Der Validierungssatz besteht ausschließlich aus dem zurückgehaltenen TrashNet-Val-Anteil und bildet damit das geplante Tischszenario ab. Der Testsatz kombiniert dmedhi-Validation sowie TACO- und Roboflow-Val/Test-Anteile. SHA-256-Prüfsummen verhindern identische Bilddateien über Splitgrenzen hinweg; sie schließen inhaltlich ähnliche Aufnahmen jedoch nicht sicher aus. Ein zeilenweises Manifest speichert für jedes Bild Zielsplit, Quelle, ursprünglichen Split, Quell-ID, Annotationen und Prüfsumme.

3.4 Geplanter kontrollierter Vergleich

Auf dem aktuellen Datensatz wird YOLO11n trainiert und anschließend in FP32- und INT8-Form evaluiert. Der Lauf ist zum Zeitpunkt dieser Fassung noch nicht abgeschlossen. Für beide Varianten bleiben Testdaten, Eingabegröße, Konfidenzregeln und Messhardware gleich. Berichtet werden mAP50–95 als Hauptmetrik, mAP50 und klassenweise AP sowie Precision, Recall und F_1 bei dokumentierter Konfidenzschwelle. Zusätzlich werden Dateigröße und CPU-Latenz nach Aufwärmritten über dieselbe Bildmenge gemessen. Dadurch lässt sich der Quantisierungseffekt als Differenz beziehungsweise Verhältnis angeben, ohne Hardware- und Datensatzeffekte zu vermischen.

Der Testsatz bleibt bis zur endgültigen Modellwahl unangetastet; Anpassungen erfolgen anhand des Validierungssatzes. Ein späterer Raspberry-Pi-Benchmark muss dasselbe exportierte Modell, dieselbe Eingabegröße und ein festes Messprotokoll verwenden. Kamera, Tracking und Roboterarm werden danach

separat untersucht, damit mechanische und softwareseitige Fehler nicht vermischt werden.

4 Implementierung

Implementiert ist ein Software-Prototyp für die reproduzierbare Datenaufbereitung, Detektion und Ergebnisdarstellung. Die physische Sortieranlage ist davon klar getrennt.

4.1 Konfiguration und Kommandozeile

Projektweite Klassen, Pfade sowie Standardwerte für Training und Inferenz liegen in `mira.yaml`. Eine modulare Kommandozeile stellt unter anderem Befehle zum Zusammenführen und Prüfen von Datensätzen, zum Trainieren, Exportieren und Evaluieren von Modellen sowie für Live-Inferenz und Dashboard bereit. Experimentparameter können in YAML-Dateien festgehalten und dadurch erneut ausgeführt werden.

4.2 Datensatzaufbereitung

Der generische Merger der Kommandozeile liest Quellen aus einem YAML-basierten Register ein. Er kann vorhandene YOLO-Daten übernehmen, Klassen-IDs abbilden und COCO-Annotationen in das YOLO-Format umwandeln. Beim Zusammenführen entstehen Trainings- und Validierungsordner sowie eine passende Dataset-YAML; ein Dry-Run und eine Strukturprüfung erlauben Kontrollen vor dem Training.

Der tatsächlich verwendete Bestand `merged_mira_balanced` wurde dagegen mit dem separaten Builder `scripts/build_balanced_dataset.py` erzeugt. Dieser lädt die vier festgelegten Quellen, wendet deren projektspezifische Klassenabbildungen und Splitregeln an, wählt die Trainingsbilder mit Seed 42 annähernd nach Boxanzahl aus, entfernt dateiidentische Bilder anhand von SHA-256-Prüfsummen und schreibt zusätzlich Testsplit und Herkunftsmanifest. Der Registry-Merger und dieser Builder sind somit zwei unterschiedliche Aufbereitungswege.

4.3 YOLO-Inferenz und Tracking

Die Live-Inferenz lädt Ultralytics-kompatible Detektoren und verarbeitet Bilder einer USB-Kamera. Ausgegeben werden Begrenzungsrahmen, Klassen und Konfidenzen. Konfidenz-, IoU- und Bildgrößenschwellen sind konfigurierbar; unsichere Treffer können in der Visualisierung gesondert markiert werden.

ByteTrack ist für PyTorch-Modelle (`.pt`) optional eingebunden und vergibt persistente Objekt-IDs über aufeinanderfolgende Frames. Für TFLite-Modelle ist Tracking im aktuellen Inferenzpfad deaktiviert. Damit ist ByteTrack Bestandteil des PT-Prototyps, aber kein nachgewiesener Teil einer Edge-Pipeline.

4.4 Dashboard-Prototyp

Das Control Center ist als FastAPI-Anwendung mit REST- und WebSocket-Schnittstellen umgesetzt. Es kann Kamerastatus, Modellwahl, Videoframes, Detektionen und Laufzeitmetriken an eine Browseroberfläche übertragen. Der aktuelle Stand ist ein Dashboard-Prototyp zur Beobachtung und Konfiguration der Inferenz, keine Steuerung einer Sortiermaschine.

4.5 Modellexport

Trainierte YOLO-Modelle können über die Pipeline nach FP32-TF Lite, INT8-TF Lite oder ONNX exportiert werden. Für den INT8-Export wird ein Kalibrierungsdatensatz angegeben. Die erzeugten Formate werden getrennt bewertet; aus einem erfolgreichen Export folgt noch keine Aussage zur Geschwindigkeit auf der Zielhardware.

4.6 Nicht implementierte Systemteile

Eine Ausführung auf Raspberry Pi, die Robotermechanik und eine serielle Verbindung zur Aktorik sind **nicht implementiert beziehungsweise ausstehend**. Entsprechend existieren derzeit weder ein geschlossener Regelkreis noch ein getestetes Handshake-Protokoll zwischen Erkennung und Roboter.

5 Ergebnisse

Berichtet werden ausschließlich bereits protokollierte Experimente. Die Klassifikation und die Objektdetektion sind verschiedene Aufgaben; ihre Kennzahlen sind daher nicht direkt vergleichbar. Ebenso gelten YOLO-Werte jeweils für den Validierungssatz des betreffenden Experiments.

5.1 Historische Klassifikationsergebnisse

Die vierklassige Klassifikation umfasste Glas, Metall, Papier und Plastik; die spätere Klasse *Trash* war nicht enthalten. Die protokollierte Validierungsgenauigkeit betrug 61.00 % für das selbst entwickelte CNN (EXP-001), 84.28 % für MobileNetV2 mit eingefrorenem Backbone (EXP-002) und 87.42 % nach Fine-Tuning (EXP-003). Da EXP-001 mit einer kleineren Datengrundlage entstand, ist diese Folge kein kontrollierter Architekturvergleich.

Tabelle 1: Protokollierte Ergebnisse der vierklassigen Klassifikation

Modell	Val.-Accuracy
EXP-001, eigenes CNN	61.00 %
EXP-002, MobileNetV2 eingefroren	84.28 %
EXP-003, MobileNetV2 Fine-Tuning	87.42 %
EXP-004, INT8 TF Lite	87.35 %

Beim Export des feinabgestimmten Klassifikators ist zwischen Zahlenformat und Container zu unterscheiden: Das FP32-TF Lite-Modell belegt 8.49 MiB, das mit 100 repräsentativen Bildern kalibrierte

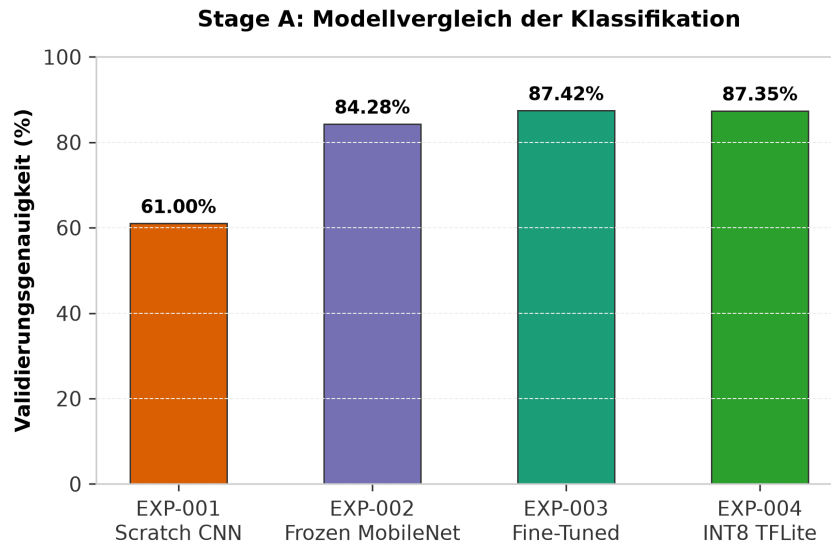


Abbildung 1: Vergleich der Validierungsgenauigkeit der vierklassigen Klassifikation (Stage A). EXP-001 verwendete eine abweichende Datengrundlage und ist deshalb kein kontrollierter Architekturvergleich.

INT8-TFLite-Modell 2.61 MiB. INT8 ist damit rund 69 % kleiner als FP32-TFLite. Für INT8 wurde eine Validierungsgenauigkeit von 87.35 % protokolliert; Laufzeitangaben werden mangels ausreichend dokumentierter, einheitlicher Messumgebung nicht verwendet.

5.2 Historische Detektionsergebnisse

Für die YOLO11n-Reihe EXP-013 bis EXP-017 sind die protokollierten PyTorch-Validierungen in Tabelle 2 zusammengefasst. Die Datensätze und Validierungssplits unterscheiden sich, besonders bei EXP-016 ohne *Trash*-Klasse. Die Tabelle dokumentiert daher einzelne Trainingsläufe und ist kein kontrollierter Architekturbenchmark.

Tabelle 2: Protokollierte Validierung der YOLO11n-PyTorch-Modelle

Exp.	Trainingsdaten	mAP50	mAP50–95
013	TACO + TrashNet	55.1 %	49.8 %
014	TACO + TrashNet + Roboflow	60.7 %	50.6 %
015	TACO + TrashNet + WaRP	56.0 %	45.1 %
016	WaRP, ohne <i>Trash</i>	58.8 %	43.2 %
017	alle vier Quellen	59.3 %	46.5 %

EXP-014 erreichte auf seinem Validierungssatz 60.7 % mAP50 und 50.6 % mAP50–95. Klassenspezifisch wurden 50.2 % für Glas, 71.3 % für Metall, 82.9 % für Papier, 72.1 % für Plastik und 26.9 % für Trash gemessen. Damit war Trash in diesem Lauf die schwächste Klasse.

Die quantisierten YOLO-Exporte sind mit etwa 2.90 MiB dokumentiert; ihnen wird hier nicht ohne separate, formatgleiche Evaluation die PyTorch-mAP zugeschrieben.

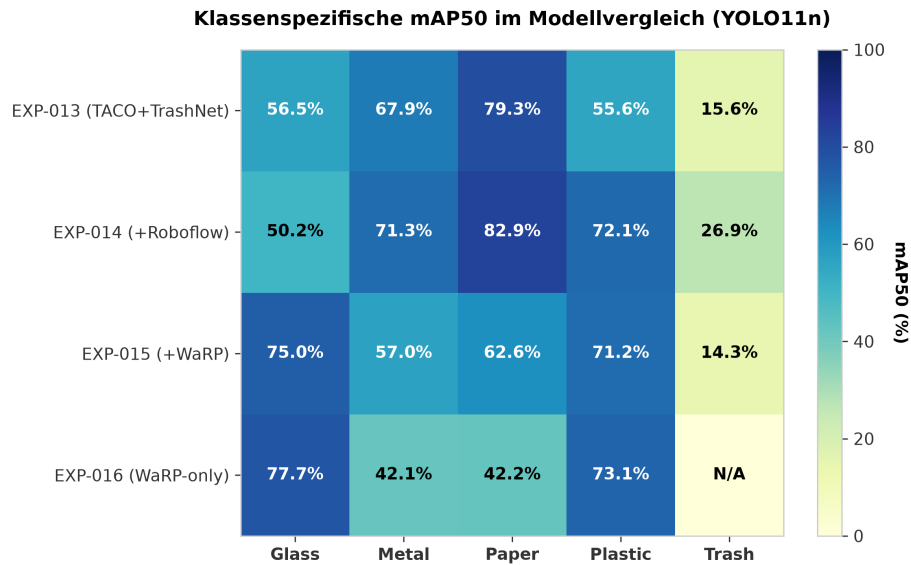


Abbildung 2: Klassenspezifische mAP50 der YOLO11n-Experimente EXP-013 bis EXP-016. EXP-016 enthält keine *Trash*-Klasse (N/A). Die Werte stammen aus den jeweiligen Experimentprotokollen und sind nicht auf einem gemeinsamen Testset gemessen.

5.3 Einordnung des Field-Benchmarks

Der vorhandene Field-Benchmark prüft auf 805 Bildern lediglich, ob eine Klasse irgendwo im Bild erkannt wurde (*image-level class presence*). Er bewertet weder die Lage noch die Überlappung der Begrenzungsrahmen und ist daher kein Detektions-mAP. Zudem sind interne Gesamtsummen und verwendete Konfidenzschwellen nicht für alle Modellformate konsistent dokumentiert. Eine modellübergreifende Rangfolge sowie FP32–INT8-Schlussfolgerungen aus diesem Benchmark werden deshalb nicht berichtet.

Messungen auf Raspberry Pi und am Gesamtsystem liegen nicht vor. Ergebnisse neuer Teacher- oder Distillationsexperimente werden erst nach abgeschlossener, reproduzierbarer Auswertung aufgenommen.

6 Diskussion

Die Experimente belegen einzelne Eigenschaften der entwickelten Software und Modelle, erlauben aber noch keine Aussage über ein vollständiges autonomes Sortiersystem. Insbesondere sind Validierungsmetriken auf kuratierten Datensätzen, Messungen auf einer Intel-CPU und Prognosen für die Zielhardware getrennt zu betrachten.

6.1 Bewertung der ursprünglichen Hypothesen

Die zu Projektbeginn formulierte Transfer-Learning-Hypothese H1 wird durch die historischen Werte allenfalls vorläufig gestützt: MobileNetV2 mit Fine-Tuning erreichte 87.42 %, das Scratch-CNN 61.00 %. EXP-001 entstand jedoch mit einer kleineren Datengrundlage. Der Abstand von 26,42 Prozentpunkten ist deshalb kein kontrollierter Architektureffekt und bestätigt H1 nicht kausal (Yosinski u. a., 2014).

H2 ist nur hinsichtlich Genauigkeit und Dateigröße des Stage-A-Klassifikators auswertbar. Das Keras-

Quellmodell umfasst 23.48 MiB. Der Export in das TFLite-Format umfasst in FP32 8.49 MiB und in INT8 2.61 MiB. Gegenüber FP32-TFLite sank die Größe durch INT8 somit um rund 69 %; die protokollierte Validierungsgenauigkeit sank um 0,07 Prozentpunkte. Die früher notierten Laufzeiten stammen nicht aus einer ausreichend dokumentierten, formatgleichen Messung und werden daher nicht zur Bestätigung einer Latenzhypothese herangezogen. Der Effekt auf den YOLO-Detektor und einen Raspberry Pi bleibt offen.

H3 ist vorläufig gestützt: Die Software kann mit YOLO11n mehrere Bounding Boxes in einem Bild ausgeben. EXP-014 erreichte auf seinem Validierungssplit 60.7 % mAP50, für *Trash* jedoch nur 26.9 %. Damit ist die technische Mehrfachdetektion gezeigt, nicht aber eine robuste Lokalisierung in unbekannten Einsatzumgebungen. E1 ist nicht umgesetzt: Der aktuelle Inferenzpfad enthält keinen EMA-Filter. Eine Wirkung zeitlicher Glättung auf mechanische Fehltrigger oder einen Sortiervorgang wurde daher nicht gemessen.

6.2 Gültigkeit der Auswertung

EXP-005 ist kein positiver Leistungsnachweis. Die automatisch erzeugten Bounding Boxes enthielten systematisch Tischkanten; die hohe interne mAP50 von 82.3 % entstand durch Data Leakage und Hintergrund-Overfitting. Das Versagen auf anderen Hintergründen zeigt, dass das Modell keine übertragbare Objektrepräsentation gelernt hatte.

Auch der als „Field Benchmark“ bezeichnete Vergleich ist in seiner jetzigen Form **kein gültiger Feldtest**: Er verwendet das `mira_v2`-Validierungsset und damit keine unabhängig im späteren Einsatz erhobenen Feldaufnahmen. Zudem ist die Auswertung mit einer einheitlichen Konfidenzschwelle für unterschiedlich kalibrierte FP32- und INT8-Modelle nicht hinreichend kontrolliert. Die dort berichteten F1-Werte, einschließlich 85.8 % für EXP-014 und des scheinbaren INT8-Rückgangs, sind daher vorläufige Pipeline-Ausgaben. Sie dürfen weder als Generalisierungsnachweis noch als Beleg praktischer Sortierqualität interpretiert werden. Erforderlich sind ein vorab festgelegtes, unabhängiges Testset, identische Zuordnungsregeln, separat kalibrierte Schwellen und eine klassenweise Fehleranalyse.

Es liegen historische Laufzeitangaben von einer Intel-Core-i7-CPU und einer NVIDIA-T4-Cloud-GPU vor; mangels eines einheitlich dokumentierten Messprotokolls eignen sie sich jedoch nicht für den aktuellen FP32-/INT8-Vergleich. Die für Raspberry Pi genannten 90 ms bis 120 ms sind Projektionen, keine Messwerte. Weder Raspberry Pi noch Kamera-Roboter-Gesamtsystem wurden vermessen; Aussagen zu Bildrate, Greifgenauigkeit, Durchsatz, Energiebedarf oder Fehlwürfen sind deshalb derzeit nicht möglich.

6.3 Klassen, Materialbegriff und Datenbias

Die fünf Zielklassen sind visuelle Entsorgungskategorien, keine verlässliche stoffliche Analyse. Eine RGB-Kamera erfasst Form, Farbe, Textur und Reflexion, aber keine chemische Zusammensetzung. Weißes zerknülltes Papier kann deshalb wie Kunststofffolie erscheinen; äußerlich ähnliche Behälter können aus verschiedenen Werkstoffen bestehen. Mehrschichtige Verbunde, beschichtetes Papier, Getränkekartons und verschmutzte Verpackungen besitzen zudem keine eindeutige Zuordnung zu genau einer Materialklasse. MIRA erkennt somit gelernte visuelle Kategorien und kann echte Materialidentität ohne zusätzliche Sensorik nicht garantieren.

Das Remapping der Quelldatensätze verschärft diese Einschränkung. Spezifische Objekt- und Materialtaxonomien wurden auf die fünf Zielklassen verdichtet.

Dabei gehen Unterschiede zwischen Objektart, Material und lokalem Entsorgungsweg verloren; Verbunde werden durch eine Projektentscheidung einer einzigen Klasse zugeordnet. Stage A enthält nur vier Klassen und muss unbekannten Restmüll zwangsläufig falsch zuordnen. In Stage B umfasst *Trash* visuell sehr verschiedene Gegenstände. Auch im aktuellen Datensatz können Quellenabhängigkeit, kanonische Objektansichten, uneinheitliche Annotationen und inhaltlich ähnliche Aufnahmen über Splitgrenzen hinweg die Metriken verzerren (Ben-David u. a., 2010). Die historischen Läufe EXP-013 bis EXP-017 sprechen dafür, dass die Datenzusammensetzung die klassenweisen Ergebnisse stark beeinflusst; wegen unterschiedlicher Datensätze und Validierungssplits bilden sie aber keine kontrollierte Ablation.

6.4 Stand der laufenden Arbeiten

Der aktuelle balancierte Datensatz ist erstellt, das darauf vorgesehene YOLO11n-Training und die formatgleiche FP32-/INT8-Auswertung sind jedoch noch nicht abgeschlossen. Teacher-, Baseline- und Distillationsläufe befinden sich ebenfalls nur in Planung oder Bearbeitung. Erwartete mAP-Werte, Modellgrößen und Raspberry-Pi-Leistungen sind Projektionen und werden nicht als Ergebnisse verwendet. Erst nach abgeschlossenem Training und Evaluation auf dem eingefrorenen Testset ist ein belastbarer Vergleich möglich.

7 Fazit und Ausblick

7.1 Fazit

MIRA demonstriert eine funktionsfähige Softwarepipeline zur Klassifikation und Mehrfachobjekterkennung von Recyclingobjekten. In Stage A erreichte MobileNetV2 mit Fine-Tuning 87.42 % Validierungsgenauigkeit; wegen der kleineren Datengrundlage von EXP-001 ist der Abstand zum Scratch-CNN jedoch kein kontrollierter Architektureffekt. Das Keras-Quellmodell umfasst 23.48 MiB, der FP32-TFLite-Export 8.49 MiB und der INT8-TFLite-Export 2.61 MiB, bei nahezu unveränderter protokollierter Validierungsgenauigkeit zwischen den beiden TFLite-Varianten. Eine belastbare Latenzaussage folgt daraus nicht. In Stage B erreichte EXP-014 auf seinem eigenen Validierungssplit 60.7 % mAP50, zeigte aber insbesondere bei *Trash* deutliche Schwächen.

Die historischen Ergebnisse stützen die ursprünglichen Hypothesen damit nur teilweise und vorläufig. Der fehlerhafte Auto-Labeling-Versuch zeigt, wie stark Leakage und Datensatzgestaltung interne Metriken verfälschen können. Die aktuelle Forschungsfrage nach dem kontrolliert gemessenen Zielkonflikt von Detektionsgüte, Modellgröße und CPU-Latenz ist noch nicht abschließend beantwortet, weil Training und formatgleiche Auswertung auf dem neuen Datensatz ausstehen. Demonstriert sind Softwarefunktionen und protokollierte Modellwerte, nicht zuverlässige Materialbestimmung, Generalisierung in einem unabhängigen Feldtest oder automatische physische Sortierung.

7.2 Ausblick

Als nächstes müssen die ausstehenden Trainingsläufe auf dem bereits erstellten balancierten Datensatz abgeschlossen und auf dem eingefrorenen Testset ausgewertet werden. Knowledge Distillation ([Hinton, Vinyals und Dean, 2015](#)) ist dabei eine zu prüfende Kompressionsmethode, noch kein Projektergebnis.

Anschließend sind reproduzierbare Latenz-, Speicher- und Energiemessungen auf dem Raspberry Pi erforderlich. Erst danach soll die geprüfte Pipeline mit Kamera und Roboterarm gekoppelt und anhand von Greiferfolg, Fehlwurfquote und Durchsatz vermessen werden. Diese Schritte entscheiden, ob sich die bisher ausschließlich softwareseitigen Befunde auf das geplante Gesamtsystem übertragen lassen.

Quellen- und Literaturverzeichnis

- Abadade, Youssef u. a. (2023). „A Comprehensive Survey on TinyML“. In: *IEEE Access* 11, S. 96892–96922. DOI: [10.1109/ACCESS.2023.3294111](https://doi.org/10.1109/ACCESS.2023.3294111).
- Ben-David, Shai u. a. (2010). „A Theory of Learning from Different Domains“. In: *Machine Learning* 79.1-2, S. 151–175.
- Bengio, Yoshua, Aaron Courville und Pascal Vincent (2013). „Representation Learning: A Review and New Perspectives“. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 35.8, S. 1798–1828.
- Deng, Lei u. a. (2020). „Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey“. In: *Proceedings of the IEEE* 108.4, S. 485–532.
- dmedhi (2026). *Garbage Image Classification/Detection*. Hugging Face. URL: <https://huggingface.co/datasets/dmedhi/garbage-image-classification-detection> (besucht am 30. 07. 2026).
- Hinton, Geoffrey, Oriol Vinyals und Jeff Dean (2015). „Distilling the Knowledge in a Neural Network“. In: *NeurIPS Workshop on Deep Learning*.
- Jocher, Glenn und Jing Qiu (2024). *Ultralytics YOLO11*. Version 11.0.0. Offizielle Software-Zitation; Ultralytics weist darauf hin, dass für YOLO11 keine formale Forschungsarbeit veröffentlicht wurde. URL: <https://github.com/ultralytics/ultralytics> (besucht am 30. 07. 2026).
- Kirillov, Alexander u. a. (2023). „Segment Anything“. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, S. 4015–4026.
- Krishnamoorthi, Raghuraman (2018). „Quantizing deep convolutional networks for efficient inference: A whitepaper“. In: *arXiv preprint*. DOI: [10.48550/arXiv.1806.08342](https://doi.org/10.48550/arXiv.1806.08342). arXiv: [1806.08342](https://arxiv.org/abs/1806.08342) [cs.LG].
- LeCun, Yann, Yoshua Bengio und Geoffrey Hinton (2015). „Deep learning“. In: *Nature* 521.7553, S. 436–444.
- Neubeck, Alexander und Luc Van Gool (2006). „Efficient Non-Maximum Suppression“. In: *International Conference on Pattern Recognition*. IEEE, S. 850–855.
- Proença, Pedro F. und Pedro Simões (2020). „TACO: Trash Annotations in Context for Litter Detection“. In: *arXiv preprint*. DOI: [10.48550/arXiv.2003.06975](https://doi.org/10.48550/arXiv.2003.06975). arXiv: [2003.06975](https://arxiv.org/abs/2003.06975) [cs.CV].
- Redmon, Joseph u. a. (2016). „You Only Look Once: Unified, Real-Time Object Detection“. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, S. 779–788.
- Roboflow Universe user jerry-jukbu (2026). *Trash Detection – trash-detection-1fj jc-uqlv1*. Exakte Datensatzidentitaet aus den im Repository gespeicherten Roboflow-Exportdateien; Export vom 05. Juli 2026 mit 2783 Bildern und dortiger Lizenzangabe CC BY 4.0. URL: <https://universe.roboflow.com/jerry-jukbu/trash-detection-1fj jc-uqlv1> (besucht am 30. 07. 2026).
- Sandler, Mark u. a. (2018). „MobileNetV2: Inverted Residuals and Linear Bottlenecks“. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, S. 4510–4520. DOI: [10.1109/CVPR.2018.00474](https://doi.org/10.1109/CVPR.2018.00474).
- TensorFlow Authors (2024). *Transfer Learning and Fine-Tuning*. Zugriff am 07. Juli 2026. URL: https://www.tensorflow.org/tutorials/images/transfer_learning.
- Thung, Gary und Mindy Yang (2016). *Classification of trash for recyclability status*. Project report. Stanford University, CS229. URL: <https://cs229.stanford.edu/proj2016/report/ThungYang-ClassificationOfTrashForRecyclabilityStatus-report.pdf> (besucht am 30. 07. 2026).

Yosinski, Jason u. a. (2014). „How transferable are features in deep neural networks?“ In: *Advances in Neural Information Processing Systems*, S. 3320–3328.

Yudin, Dmitry u. a. (2024). „Hierarchical waste detection with weakly supervised segmentation in images from recycling plants“. In: *Engineering Applications of Artificial Intelligence* 128. Die offizielle WaRP-Datensatzseite bittet um Zitation dieser Veröffentlichung., S. 107542. DOI: [10.1016/j.engappai.2023.107542](https://doi.org/10.1016/j.engappai.2023.107542). URL: <https://github.com/AIRI-Institute/WaRP> (besucht am 30.07.2026).

Unterstützungsleistungen

- **Anthropic, Claude, KI-Sprachmodell:** Art der Unterstützung: Code-Review, Fehlersuche und Debugging sowie Unterstützung beim Entwerfen, Formulieren und kritischen Überarbeiten von Text und Software. Vorschläge und Ausgaben wurden vom Verfasser geprüft und bei Bedarf verändert.
- **OpenAI, KI-gestützte Werkzeuge (einschließlich ChatGPT/Codex):** Art der Unterstützung: Code-Review, Fehlersuche und Debugging sowie Unterstützung beim Entwerfen, Formulieren und kritischen Überarbeiten von Text und Software. Vorschläge und Ausgaben wurden vom Verfasser geprüft und bei Bedarf verändert.
- **TODO: [Vor- und Nachname der betreuenden Lehrkraft], Betreuungslehrkraft, Gymnasium Mülheim an der Ruhr, Mülheim an der Ruhr:** Art der Unterstützung: Fachliche Begleitung und Beratung bei Themenfindung und Strukturierung der schriftlichen Arbeit. **Vor Abgabe vervollständigen.**
- **Sparkassenstiftung Mülheim an der Ruhr, Mülheim an der Ruhr:** Stiftung; Art der Unterstützung: Sponsoring-Anträge für mechatronische Hardwarekomponenten eingereicht.

Die Datenerfassung, Ausführung der Trainingsläufe und Bewertung der Resultate lagen beim Verfasser. KI-Werkzeuge dienten als unterstützende Arbeitsmittel; die Verantwortung für Auswahl, Prüfung und Darstellung der Inhalte verbleibt beim Verfasser. Die Unterstützungsleistungen sind zusätzlich vollständig in der Jufo-Wettbewerbsverwaltung einzutragen.